

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Факультет информационных технологий
Кафедра Инфокогнитивные технологии
9.03.01 «Информатика и вычислительная техника»
Образовательная программа (профиль) «Веб-технологии»

Отчет по курсовой работе по дисциплине «Веб-разработка»
Тема: «API для чат-бота по сдаче академической разницы»

Выполнил:

Студент группы 241-3210

_____ Хужоков А.Ж.

подпись, дата

Принял:

Старший преподаватель кафедры ИКТ

_____ Даньшина М.В.

подпись, дата

Старший преподаватель кафедры ИКТ

_____ Голубева И.В.

подпись, дата

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. Анализ и проектирование	4
1.1. Анализ предметной области	4
1.2. Анализ аналогов	4
1.3. Описание функциональности проекта	5
2. Практическая реализация	6
2.1. Проектирование и реализация базы данных	6
2.2. Настройка административной панели	8
2.3. Реализация API с помощью Django REST Framework	11
2.4. Реализация CRUD-операций на страницах проекта (MVT)	13
2.5. Реализация дополнительных требований	15
ЗАКЛЮЧЕНИЕ	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18

ВВЕДЕНИЕ

В современных образовательных учреждениях студенты часто сталкиваются с необходимостью ликвидации академической разницы при переводе с одной образовательной программы на другую, при восстановлении или переходе из другого вуза. Этот процесс связан со значительным объемом документации, необходимостью координации с учебным отделом и преподавателями, а также строгим соблюдением сроков. Отсутствие единой цифровой системы для управления этим процессом создает неудобства как для студентов, так и для сотрудников университета.

Данный проект посвящен разработке бэкенда (API) для системы, автоматизирующей процесс управления академической разницей. Основным интерфейсом для взаимодействия со студентами предполагается чат-бот в мессенджере Telegram, который будет использовать разработанный API. Это позволит студентам в удобной форме отслеживать свои задолженности, загружать необходимые документы и получать уведомления о статусе их ликвидации.

Цель проекта: разработать серверную часть (API) на базе фреймворка Django REST Framework для веб-приложения и чат-бота, предназначенного для управления процессом сдачи академической разницы студентами.

Задачи проекта:

- Проанализировать предметную область и существующие аналоги.
- Спроектировать структуру базы данных, реализовать модели и связи.
- Разработать и детально настроить административный интерфейс Django.
- Реализовать API с использованием Django REST Framework, включая фильтрацию, поиск, пагинацию и обработку сложных запросов.
- Реализовать базовые веб-страницы для демонстрации CRUD-операций.
- Интегрировать дополнительные библиотеки для логирования истории, экспорта данных и других функций.
- Настроить запуск проекта в Docker-контейнерах для обеспечения переносимости.

Ссылка на репозиторий проекта: <https://github.com/askerdev/academic-difference-api>

1. Анализ и проектирование

1.1. Анализ предметной области

Предметная область — организация процесса ликвидации академической разницы в учебном заведении. Академическая разница представляет собой список дисциплин и форм отчетности (экзамен, зачет), которые студент должен сдать, чтобы соответствовать текущему учебному плану.

Ключевые процессы:

- **Формирование списка задолженностей:** Учебный отдел формирует индивидуальный список дисциплин для студента.
- **Процесс сдачи:** Студент связывается с преподавателями, выполняет задания и сдает необходимые формы контроля.
- **Отчетность:** Преподаватель фиксирует результат, который затем передается в учебный отдел.
- **Контроль сроков:** Процесс ликвидации разницы ограничен по времени.

Цифровизация этого процесса призвана решить проблемы потери информации, долгого ожидания и отсутствия прозрачности для студента.

1.2. Анализ аналогов

1.2.1. Чат-бот Московского Политеха для сдачи РУП

Существующий бот, предназначенный для помощи студентам в сдаче Расхождений в Учебных Планах (РУП). Хотя бэкенд не был доступен для анализа, структура его базы данных известна. Функционал включает регистрацию, загрузку файлов РУП и систему уведомлений. Этот аналог подтверждает востребованность автоматизации подобных процессов через чат-ботов.

1.2.2. Телеграм-бот НИ ТГУ (@TSU_Expecto_Patronum_bot)

Комплексный бот для студентов Томского государственного университета. Он предоставляет доступ к расписанию, настройкам уведомлений о дедлайнах и задолженностях, а также имеет функцию анонимной обратной связи. Для проектируемой системы интересен подход к гибкой настройке уведомлений и интеграции с существующими университетскими сервисами (LMS).

1.3. Описание функциональности проекта

Проектируемая система имеет три роли пользователей:

- **Студент:** Взаимодействует с системой через чат-бота. Может просматривать свои академические задолженности, сроки их сдачи, информацию о преподавателях, а также настраивать уведомления.
- **Преподаватель:** Имеет доступ к административной панели в режиме «только для чтения». Может просматривать информацию о студентах и их задолженностях по своим дисциплинам, а также формировать отчеты.
- **Администратор:** Обладает полными правами в системе через административную панель. Может создавать, редактировать и удалять любые данные: студентов, преподавателей, дисциплины, академические группы и задолженности.

2. Практическая реализация

2.1. Проектирование и реализация базы данных

На основе анализа предметной области были спроектированы инфологическая (Рисунок 1) и даталогическая (Рисунок 2) схемы базы данных.

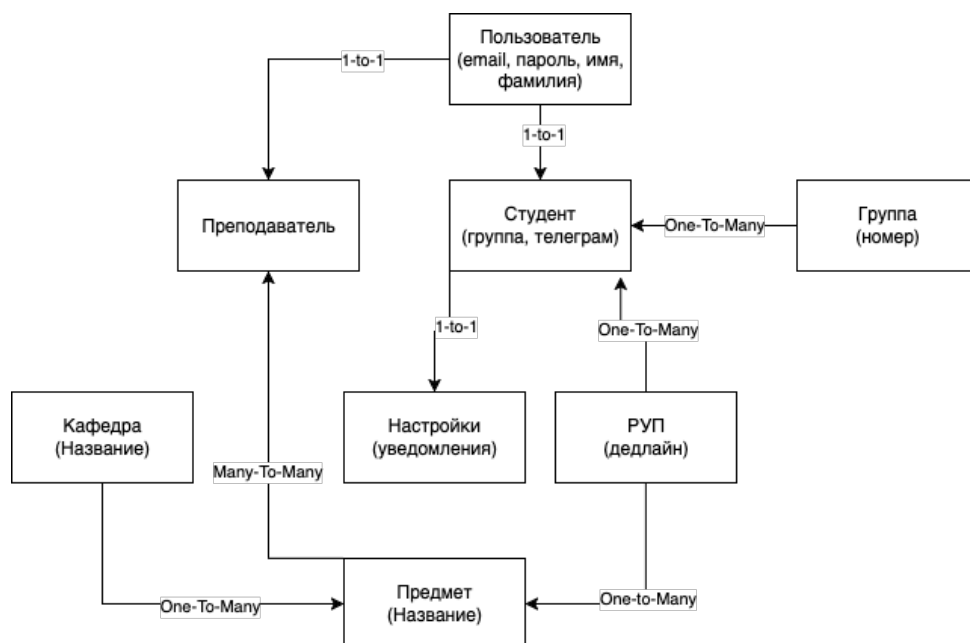


Рис. 1. Инфологическая схема базы данных

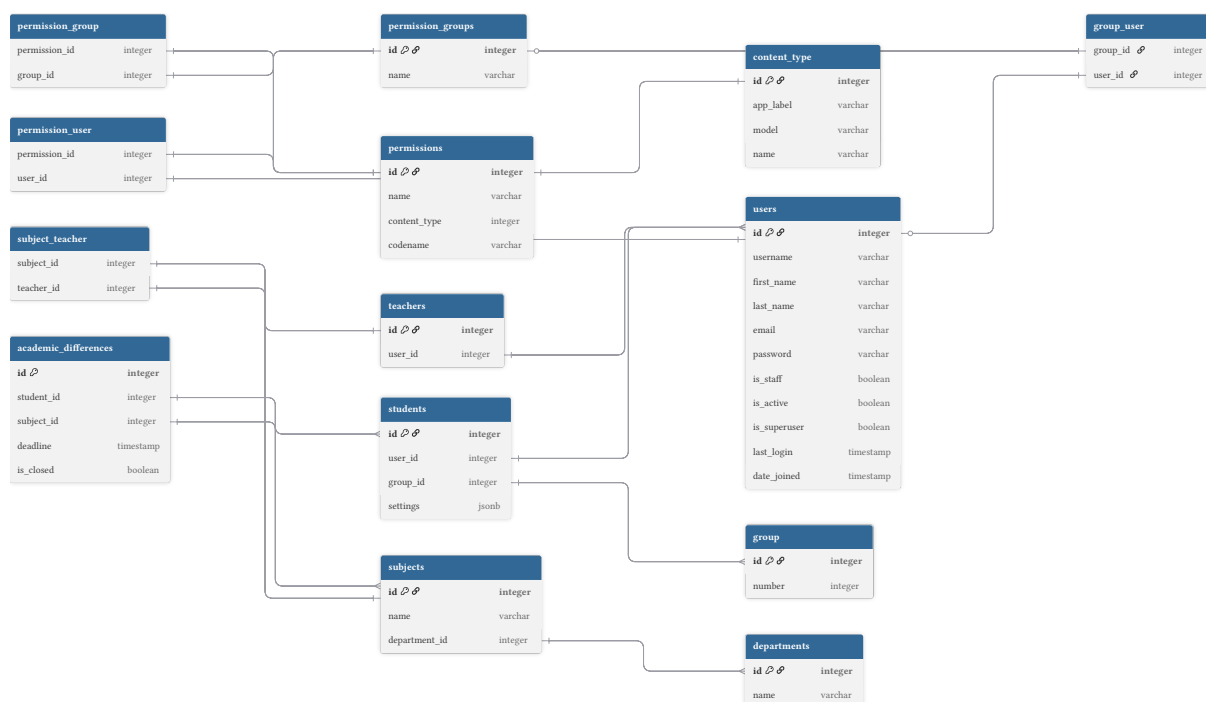


Рис. 2. Даталогическая схема базы данных

Структура БД реализована с помощью Django ORM. В проекте определены 6 основных моделей: AcademicGroup, Student, Department, Subject, Teacher

и AcademicDifference. Для избежания дублирования кода была создана абстрактная модель Common, содержащая поля created_at и updated_at.

Для полей моделей активно используются verbose_name для улучшения читаемости в административной панели. Во всех моделях переопределен метод __str__ для информативного отображения объектов. Пример реализации модели Student:

```
// api/models.py
class Student(Common):
    user = models.OneToOneField(
        User, on_delete=models.PROTECT, verbose_name="Related
user"
    )
    group = models.ForeignKey(
        AcademicGroup, on_delete=models.PROTECT,
verbose_name="Related group"
    )
    telegram_id = models.BigIntegerField(
        unique=True, verbose_name="Telegram ID"
    )
    settings = JSONField(default=dict, blank=True,
verbose_name="User Settings")

    class Meta:
        verbose_name = "student"
        verbose_name_plural = "students"

    def __str__(self):
        return (
            f"{self.user.last_name} {self.user.first_name}
{self.group.number}"
        )
```

Связь «многие ко многим» реализована в модели Teacher для привязки преподавателей к нескольким предметам:

```
// api/models.py
class Teacher(Common):
    user = models.OneToOneField(User, on_delete=models.PROTECT)
    subjects = models.ManyToManyField(
        Subject, verbose_name="Teacher To Subject"
```

```
)  
# ...
```

2.2. Настройка административной панели

Административная панель Django была значительно кастомизирована для удобного управления данными. Был создан миксин `AdminMixin` для общих настроек.

- **list_display:** Для всех моделей настроен вывод ключевых полей в списке объектов. Для модели `AcademicDifference` добавлен кастомный метод `view_department` с декоратором `@admin.display` для вывода кафедры, к которой относится предмет задолженности.
- **inlines:** На странице `AcademicGroup` встроено редактирование списка студентов этой группы (`StudentInline`), что позволяет управлять составом группы напрямую (Рисунок 3).

```
// api/admin.py  
class StudentInline(admin.TabularInline):  
    model = Student  
  
@admin.register(AcademicGroup)  
class AcademicGroupAdmin(AdminMixin):  
    inlines = (StudentInline,)
```

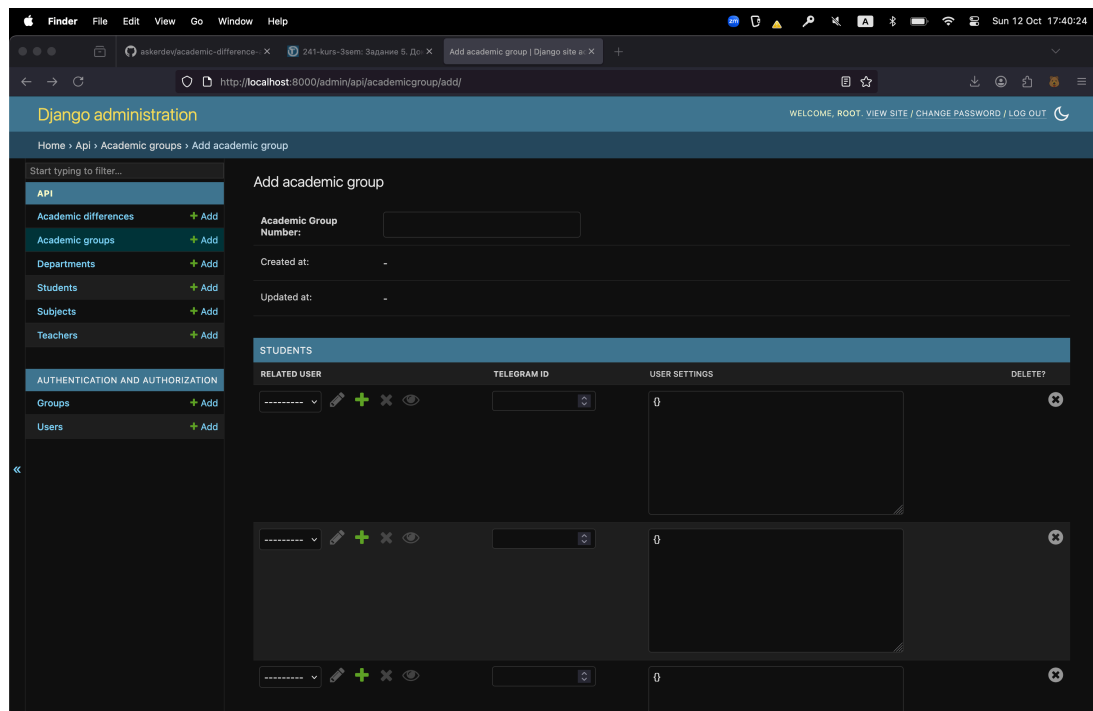


Рис. 3. Инлайн-редактирование студентов на странице группы

- **Фильтрация и поиск:** Для удобной навигации добавлены `list_filter` и `search_fields`. Например, в `StudentAdmin` можно искать по имени, фамилии, юзернейму, email, номеру группы и `telegram_id`.
- **Оптимизация работы с внешними ключами:** `raw_id_fields` и `autocomplete_fields` используются для полей `ForeignKey`, чтобы избежать загрузки тысяч записей в выпадающий список. `filter_horizontal` применен для удобного выбора предметов у преподавателя (Рисунок 4).

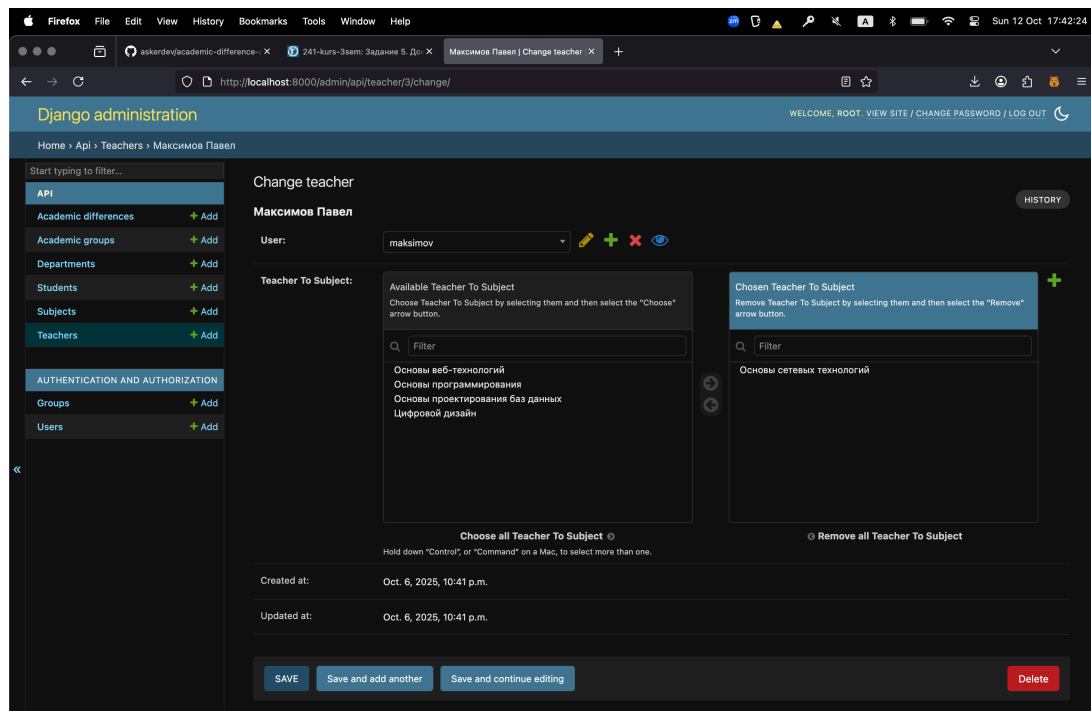


Рис. 4. Выбор предметов для преподавателя с помощью `filter_horizontal`

- **django-import-export:** Для всех моделей подключена возможность экспорта данных в форматах CSV, XLS, XLSX. Для `AcademicDifferenceAdmin` переопределен метод `get_export_queryset` для экспорта только незакрытых задолженностей.

```
// api/admin.py
@admin.register(AcademicDifference)
class AcademicDifferenceAdmin(AdminMixin):
    def get_export_queryset(self, request):
        return
        AcademicDifference.objects.filter(is_closed=False)
    # ...
```

- **django-simple-history:** Подключена библиотека для ведения истории изменений всех объектов. В админ-панели для каждой записи доступна кнопка «History», позволяющая отследить, кто и когда вносил изменения (Рисунок 5).

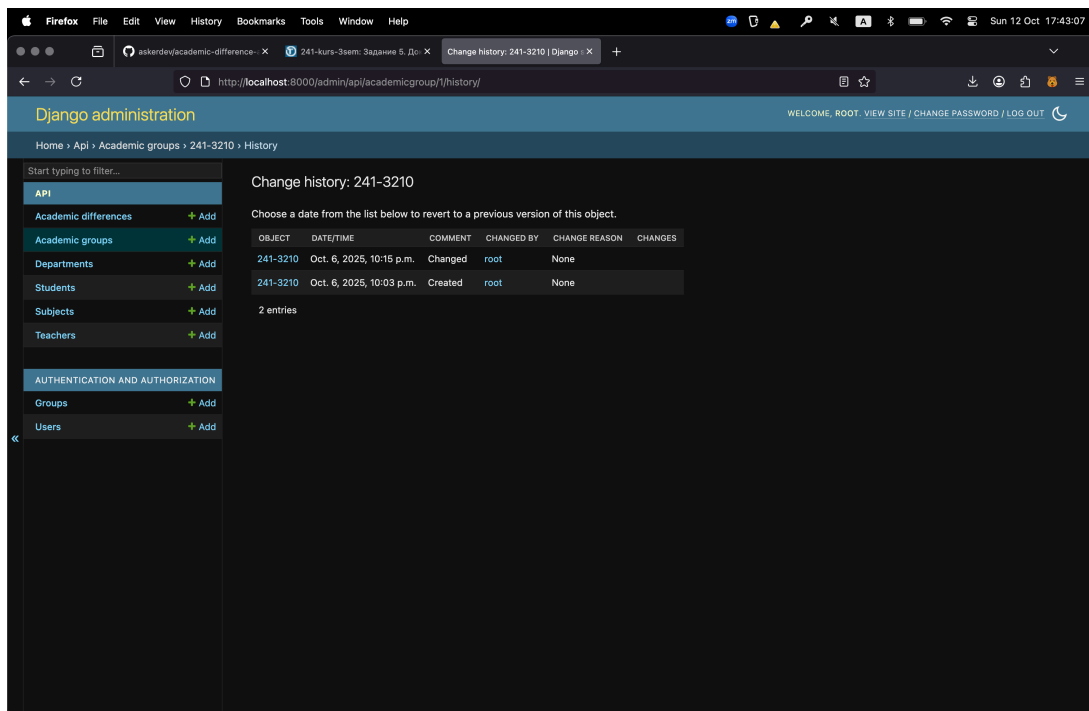


Рис. 5. Просмотр истории изменений объекта

2.3. Реализация API с помощью Django REST Framework

Основной функционал проекта реализован в виде REST API. Для каждой модели создан `ViewSet` на основе `viewsets.ModelViewSet`, который предоставляет стандартные CRUD-операции.

- **Фильтрация и поиск:** Реализовано несколько механизмов фильтрации:
 - `SearchFilter`: базовый поиск по полям, указанным в `search_fields` (например, поиск студентов по ФИО).
 - `DjangoFilterBackend` с кастомным `FilterSet`: для модели `AcademicDifference` создан класс `AcademicDifferenceFilterset`, позволяющий гибко фильтровать задолженности по параметрам в URL (например, по имени студента, фамилии преподавателя, диапазону дат дедлайна).
 - Фильтрация по текущему пользователю: в `AcademicDifferenceViewSet` метод `get_queryset` переопределен так, что обычный пользователь видит только свои задолженности, а суперпользователь — все.

```
// api/views/rest.py
class AcademicDifferenceViewSet(viewsets.ModelViewSet):
    # ...
    filterset_class = AcademicDifferenceFilterset
```

```
def get_queryset(self):
    if self.request.user.is_superuser:
        return AcademicDifference.objects.all()
    return AcademicDifference.objects.filter(
        student__user=self.request.user
    )
```

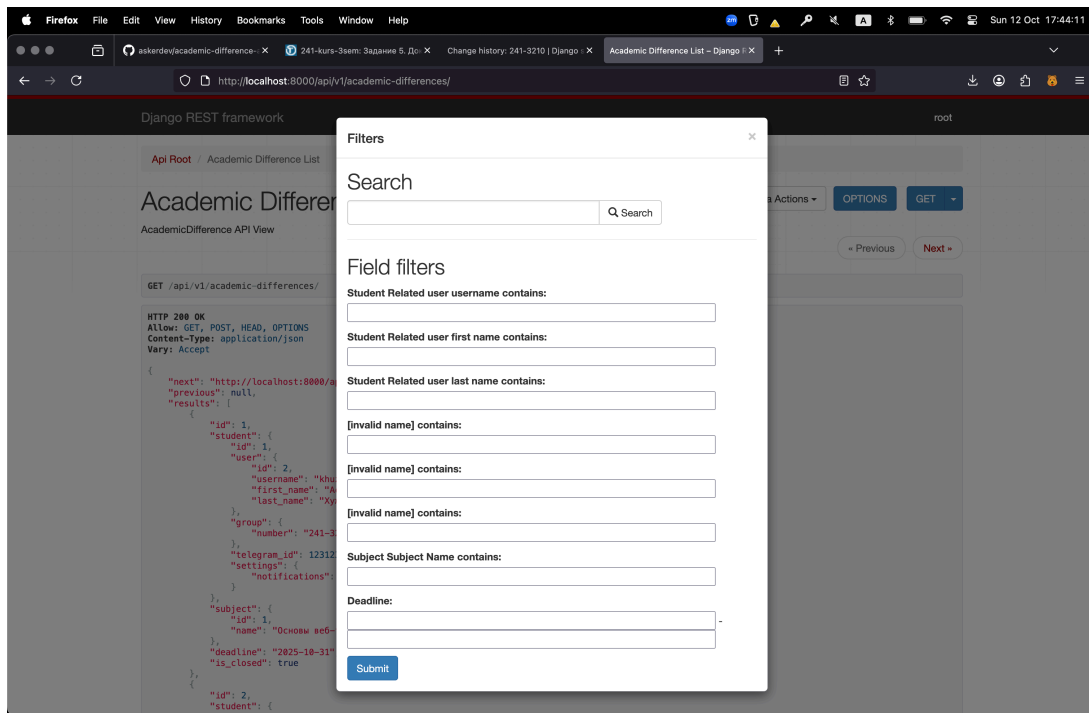


Рис. 6. Интерфейс Browsable API с доступными фильтрами

- **Сложные запросы с Q:** Для реализации нетривиальной бизнес-логики используются Q-объекты. Например, в экшне `upcoming_for_teacher` выполняется запрос для получения незакрытых задолженностей у студентов определенных групп, закрепленных за текущим преподавателем.

```
// api/views/rest.py
@action(methods=["GET"], detail=False)
def upcoming_for_teacher(self, request):
    differences = AcademicDifference.objects.filter(
        (
            Q(subject__teacher__user=request.user)
            if not request.user.is_superuser
            else Q()
        )
    )
```

```

        & (
            Q(student__group__number="241-3210")
            | Q(student__group__number="241-322")
        )
        & Q(is_closed=False)
    )
    # ...

```

- **Пользовательские actions:**

- `@action(methods=['POST'], detail=True)`: реализован экшн `close` для закрытия конкретной задолженности. `detail=True` означает, что экшн применяется к одному объекту.
- `@action(methods=['GET'], detail=False)`: реализован экшн `upcoming` для получения списка всех предстоящих (непросроченных) задолженностей. `detail=False` означает, что экшн применяется ко всему списку.

- **Валидация:** В `StudentSerializer` реализован метод `validate_settings`, который проверяет корректность JSON-объекта с настройками пользователя, обеспечивая наличие и валидность поля `notifications`.

```

// api/serializers.py
class StudentSerializer(serializers.ModelSerializer):
    # ...
    def validate_settings(self, value):
        settings = json.loads(value)
        if "notifications" not in settings:
            raise serializers.ValidationError("...")
        if settings["notifications"] not in (True, False):
            raise serializers.ValidationError("...")
        return value

```

- **Пагинация:** Ко всем `ViewSet`'ам подключена стандартная пагинация `PageNumberPagination`.

2.4. Реализация CRUD-операций на страницах проекта (MVT)

В дополнение к API, для демонстрации работы с моделями в рамках классического MVT-подхода Django, были созданы веб-страницы для управления студентами (`api/views/mvt.py`). Реализованы следующие функции:

- Просмотр списка всех студентов (`list_students`).

- Создание нового студента (`create_student`) с использованием `django.forms.Form`.
- Редактирование данных студента (`edit_student`).
- Удаление студента (`remove_student`).

Все операции с базой данных, включающие несколько шагов (например, создание `User` и `Student`), обернуты в `transaction.atomic` для обеспечения целостности данных.

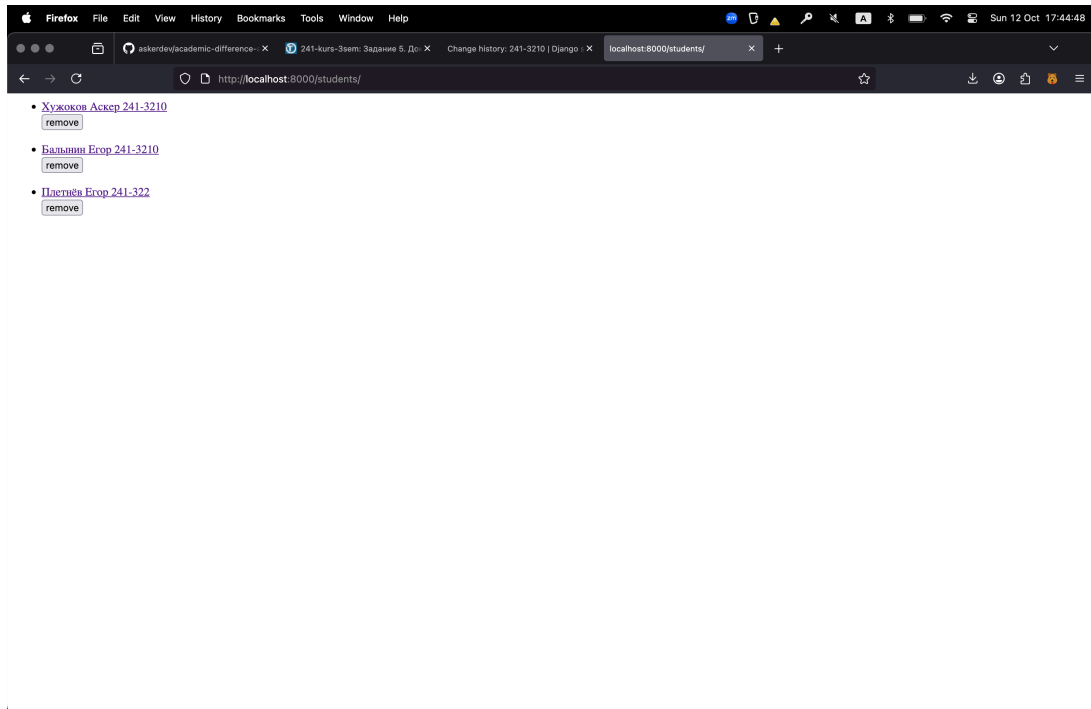


Рис. 7. Страница со списком студентов (MVT)

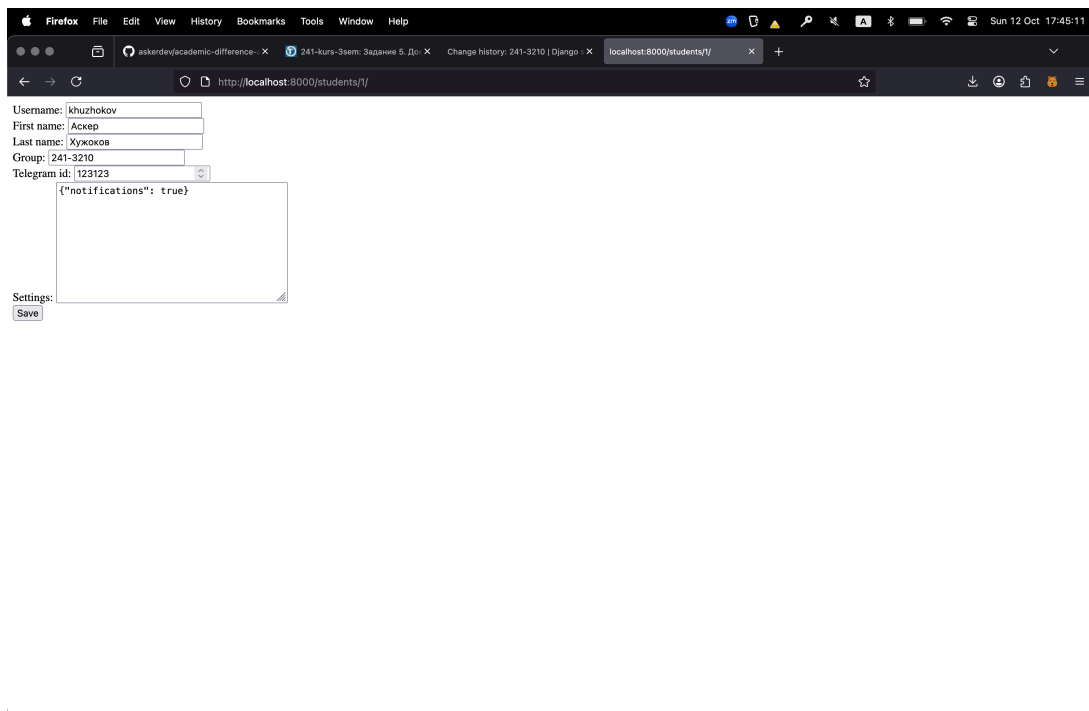


Рис. 8. Страница редактирования студента (MVT)

2.5. Реализация дополнительных требований

- **Management-команда:** Создана команда `auto_createsuperuser` (`api/management/commands/auto_createsuperuser.py`), которая позволяет создавать суперпользователя с заданными учетными данными из командной строки. Это упрощает первоначальную настройку проекта.
- **Linter:** В проекте используется связка `black` и `isort` для автоматического форматирования кода, что обеспечивает его единообразие и читаемость. Настройки заданы в файле `pyproject.toml`.
- **Docker:** Проект полностью контейнеризирован. `Dockerfile` использует многоэтапную сборку (`multi-stage build`) для уменьшения размера итогового образа. `compose.yml` описывает запуск двух сервисов: `postgres` (база данных) и `academic-api` (приложение Django). При старте контейнера автоматически применяются миграции и создается суперпользователь с помощью ранее описанной `management`-команды.

```
// compose.yml
services:
  postgres: # ...
  academic-api:
    build:
      context: .
```

```
dockerfile: ./infra/Dockerfile
command: |
  sh -c "python manage.py migrate --noinput
        python manage.py auto_createsuperuser ...
        python manage.py runserver 0.0.0.0:8000"
# ...
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была успешно разработана серверная часть (API) для системы управления академической разницей. Поставленная цель достигнута, все задачи выполнены в полном объеме.

Создан многофункциональный бэкенд на Django и Django REST Framework, включающий проработанную модель данных, детально настроенную административную панель и гибкий API с механизмами фильтрации, поиска и валидации. Реализованы все обязательные и дополнительные критерии курсовой работы, включая использование Q-объектов, кастомных actions, django-simple-history, django-import-export, management-команд, а также полную контейнеризацию проекта с помощью Docker.

Разработанное решение является надежной и масштабируемой основой для создания клиентских приложений, таких как Telegram-бот, и может быть внедрено в реальный образовательный процесс для его упрощения и автоматизации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация Django [Электронный ресурс]. URL: <https://docs.djangoproject.com/en/5.0/> (дата обращения: 20.05.2024).
2. Официальная документация Django REST framework [Электронный ресурс]. URL: <https://www.django-rest-framework.org/> (дата обращения: 20.05.2024).
3. Руководство по Django для начинающих
4. Документация django-import-export [Электронный ресурс]. URL: <https://django-import-export.readthedocs.io/> (дата обращения: 21.05.2024).
5. Документация django-simple-history [Электронный ресурс]. URL: <https://django-simple-history.readthedocs.io/> (дата обращения: 21.05.2024).
6. Документация Docker [Электронный ресурс]. URL: <https://docs.docker.com/> (дата обращения: 22.05.2024).